ReconX - Attack Surface Management Framework

Complete Documentation & Roadmap

Version: 1.0 (Current Release)
Last Updated: February 10, 2026
Status: Production-Ready CLI | Enterprise Features In Development

---

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Current Implementation Status
4. Installation & Setup
5. Configuration
6. Usage Guide
7. Database Schema
8. Four-Phase Reconnaissance Engine
9. API Documentation (Planned for Phase A)
10. Web Dashboard (Planned for Phase A)
11. Roadmap: 4-Phase Development Plan
12. Use Cases
13. Best Practices
14. Troubleshooting
15. Contributing

Executive Summary

ReconX is an enterprise-grade Attack Surface Management (ASM) platform designed to discover, inventory, and monitor an organization's digital assets and their vulnerabilities. It orchestrates **50+ reconnaissance and vulnerability scanning tools** across four distinct phases to provide comprehensive attack surface visibility.

Key Capabilities (Current + Planned)

Capability	Status	Timeline
Multi-tool Orchestration	✅ Working	Now
Phase-based Execution	✅ Working	Now
SQLite Storage	✅ Working	Now
CLI Reporting	✅ Working	Now
REST API	🔄 Planned	Phase A

Capability	Status	Timeline
Web Dashboard	🔄 Planned	Phase A
PDF/HTML Reports	🔄 Planned	Phase A
Docker Deployment	🔄 Planned	Phase A
Scheduled Scanning	🔄 Planned	Phase B
Slack/Email Alerts	🔄 Planned	Phase B
Vulnerability Tracking	🔄 Planned	Phase B
Multi-Tenant RBAC	🔄 Planned	Phase C
Jira Integration	🔄 Planned	Phase C
Compliance Frameworks	🔄 Planned	Phase C
PostgreSQL Support	🔄 Planned	Phase D
Distributed Workers	🔄 Planned	Phase D
Plugin System	🔄 Planned	Phase D

Architecture Overview

High-Level System Design

```
┌─────────────────────────────────────────────────────────────┐
│                    User Interface Layer                     │
│                                                             │
│  Future: Web Dashboard (React/Next.js)                      │
│  Current: CLI (Python) + Query Tool                         │
└─────────────────────────────────────────────────────────────┘
                  │
                  ▼
┌─────────────────────────────────────────────────────────────┐
│                   API Layer (Future)                        │
│                                                             │
│  REST API (FastAPI)                                         │
│  - Scan Management                                          │
│  - Data Retrieval                                           │
│  - Report Generation                                        │
│  - Configuration                                            │
└─────────────────────────────────────────────────────────────┘
                  │
                  ▼
┌─────────────────────────────────────────────────────────────┐
│              Orchestration & Execution Layer                │
│                                                             │
│  ReconX Orchestrator (Python)                               │
│  - Phase Management                                         │
│  - Threading Control                                        │
```

```
│  - Output Parsing                                        │
│                                                           │
│  Bash Modules (4 Phases)                                 │
│  - Discovery (50+ tools)                                 │
│  - Intelligence (30+ tools)                              │
│  - Content Discovery (20+ tools)                         │
│  - Vulnerability Scanning (20+ tools)                    │
└──────────────────────────────────────────────────────────┘
                  │
                  │
┌─────────────────▼────────────────────────────────────────┐
│              Data Storage & Processing Layer             │
│                                                           │
│  SQLite (Current) → PostgreSQL (Future Phase D)         │
│  - scan_progress                                         │
│  - subdomains                                            │
│  - ports                                                 │
│  - urls                                                  │
│  - vulnerabilities                                       │
│  - leaks                                                 │
│  - infrastructure                                        │
│                                                           │
│  Parsers (Python)                                        │
│  - JSON/XML/Text conversion                              │
│  - Structured data extraction                            │
└──────────────────────────────────────────────────────────┘
                  │
                  │
┌─────────────────▼────────────────────────────────────────┐
│            External Tools & Data Sources                 │
│                                                           │
│  Subdomain Enumeration    | Port Scanning             |
│  - Amass, Subfinder       | - RustScan, Nmap         |
│  - Assetfinder            | - ServiceVersion Detection|
│  - Sublist3r, crt.sh      |                           |
│                           |                           |
│  Vulnerability Scanning   | OSINT & Infrastructure   |
│  - Nuclei                 | - Shodan, Censys         |
│  - Dalfox (XSS)           | - SecurityTrails         |
│  - SQLMap (SQLi)          | - Whois, DNS lookup      |
│  - Corsy (CORS)           | - ASN expansion          |
│                           |                           |
│  Content Discovery        | Other Tools              |
│  - GAU, Wayback           | - Gitleaks, Subjack      |
│  - Katana, SpideyX        | - Testssl.sh, SSL tools  |
│  - FFUF, Feroxbuster      | - Cloud enum tools       |
│  - Path/File Brute-force  | - And many more...       |
└──────────────────────────────────────────────────────────┘
```

Data Flow

```
```

```
Target Domain
     │
     ▼
Phase 1: Discovery
  ├─ Run 20+ enumeration tools
  ├─ Resolve DNS
  ├─ Validate live hosts
  └─ Output: subdomains.txt, alive_hosts.txt, *.json

Phase 1 Results
     │
     ▼
Phase 2: Intelligence
  ├─ Port scanning (RustScan + Nmap)
  ├─ OSINT (Shodan, Censys)
  ├─ Takeover detection
  ├─ Repository leak scanning
  └─ Output: ports.xml, takeover.txt, leaks.json

Combination of results
     │
     ▼
Phase 3: Content Discovery
  ├─ Archive crawling (Wayback, GAU)
  ├─ Directory brute-forcing
  ├─ JavaScript analysis
  ├─ API discovery
  └─ Output: urls.txt, paths.json, js_secrets.txt

All discovered assets
     │
     ▼
Phase 4: Vulnerability Scanning
  ├─ Template-based scanning (Nuclei)
  ├─ Web app testing (XSS, SQLi, CORS)
  ├─ SSL/TLS analysis
  ├─ Specific CMS scanning
  └─ Output: vulns.json, findings.txt

All results
     │
     ▼
SQLite Database
     │
     ├─ Subdomains: 1000s records
     ├─ URLs: 1000s-10000s records
     ├─ Open Ports: 100s records
     ├─ Vulnerabilities: 10s-1000s records
     └─ Infrastructure: detailed metadata
     │
     ▼
Query Interface
```

```
    ├─ CLI: query.py
    ├─ SQL: Direct queries
    ├─ API: /api/v1/* (Future Phase A)
    └─ Dashboard: Web UI (Future Phase A)

  Reports & Export
    ├─ CSV
    ├─ JSON
    ├─ HTML (Future)
    └─ PDF (Future)
```

Current Implementation Status

✅ What's Fully Implemented

Core Engine:

- Multi-threaded target scanning
- Four-phase modular execution
- 50+ tool orchestration
- Per-target output directories
- Phase completion tracking
- Resume on failure capability
- Continue-on-fail error resilience

Data Storage:

- SQLite database with WAL mode
- Thread-safe connections
- Proper indexing for performance
- Singleton pattern for DB access
- Bulk insert optimization

Parsers:

- Subdomain enumeration outputs
- Nmap XML parsing
- RustScan output parsing
- HTTPx JSON parsing
- DNSx JSON parsing
- Nuclei JSON parsing
- Dalfox output parsing
- SQLMap output parsing
- Corsy output parsing
- Gitleaks JSON parsing
- URL discovery (gau, waybackurls, hakrawler, gospider, katana)

CLI Tools:

- `reconx.py`: Full orchestrator with all flags
- `query.py`: Target summaries, filtering, CSV export

Configuration:

- 100+ environment variables for fine-tuning
- Per-phase timeout controls
- Per-tool thread configuration
- Graceful tool absence handling

Reporting:

- Summary statistics by target
- Subdomain export (CSV)
- URL export (CSV)
- Port listing (CSV)
- Vulnerability listing (CSV)
- Severity-based filtering

🔄 Partially Implemented

- Tool run tracking database (`tool_runs` table created but not populated)
- Infrastructure data storage (table exists, data not collected)
- Acquisition tracking (table exists, not used)

❌ Known Issues & Gaps

Issue	Impact	Workaround	Fix Timeline
`config.yaml` not loaded	Settings ignored	Use env vars instead	Phase A
FFUF JSON parsing broken	No FFUF results in DB	Manual review FFUF JSON	Phase A
HTTPx results don't update alive status	Subdomain status stale	Run HTTPx again, parse separately	Phase A
Many tool outputs not parsed	Lost data	Future: Phase B parsers	Phase B
No web interface	CLI-only access	Use query.py reports	Phase A
No API	Automation impossible	SSH to run commands manually	Phase A
No scheduling	Manual re-runs needed	Cron job wrapper	Phase B
No alerts	Miss new findings	Manual log review	Phase B
No multi-user	No team collaboration	SSH access sharing	Phase C

Issue	Impact	Workaround	Fix Timeline
No role-based access	Permission issues	Filesystem permissions	Phase C

Installation & Setup

System Requirements

- **OS:** Linux (Kali Linux, Ubuntu 20.04+, Debian, or similar)
- **Python:** 3.11+
- **Disk Space:** 10GB minimum (for tools + results)
- **RAM:** 4GB minimum (8GB+ recommended for large scans)
- **Network:** Outbound to scanning targets + API endpoints

Quick Start (3 Commands)

```
# 1. Clone or navigate to repo
cd /path/to/kali-linux-asm

# 2. Run quick setup
bash setup.sh

# 3. Start scanning
python3 reconx.py -t example.com
```

Detailed Installation

Step 1: System Dependencies

```
# Install via setup.sh (installs minimal dependencies)
bash setup.sh

# OR manually install key packages
sudo apt update
sudo apt install -y git curl jq python3 python3-pip dnsutils

# Verify Python version
python3 --version  # Should be 3.11+
```

Step 2: Tool Installation

```
# Full tool installation (takes 20-30 minutes)
sudo bash install.sh
```

```
# This installs:
# - Subdomain enumeration tools (Amass, Subfinder, Assetfinder, etc.)
# - Port scanning tools (RustScan, Nmap)
# - Vulnerability scanners (Nuclei, Dalfox, SQLMap)
# - Content discovery tools (FFUF, Feroxbuster, Katana)
# - And 40+ more tools total
```

Step 3: Python Dependencies

```
pip3 install -r requirements.txt

# Packages installed:
# - requests: HTTP library
# - beautifulsoup4: HTML parsing
# - dnspython: DNS queries
# - pyyaml: Config parsing
# - colorama: Terminal colors
# - tqdm: Progress bars
# - aiohttp: Async HTTP
# - lxml: XML parsing
# - tabulate: Table formatting
```

Step 4: API Keys (Optional)

```
# Create .env file with your API keys
cp .env.example .env
nano .env

# Add:
export SHODAN_API_KEY="your_key_here"
export CENSYS_API_ID="your_id"
export CENSYS_API_SECRET="your_secret"
export GITHUB_TOKEN="your_token"
export SECURITYTRAILS_API_KEY="your_key"

# Load environment
source .env
```

Docker Setup (Future Phase A)

```
# Build and run with Docker
docker-compose up -d

# Services running:
```

```
# - API: http://localhost:8000
# - Dashboard: http://localhost:3000
# - Database: SQLite in volume ./data/
```

Configuration

Method 1: Environment Variables (Current)

All settings can be controlled via environment variables. Set before running:

```
# Core settings
export RECONX_PHASE_TIMEOUT=3600          # Seconds per phase
export RECONX_THREADS=5                   # Concurrent targets
export RECONX_CONTINUE_ON_FAIL=1          # Continue if tool fails

# Phase 1: Discovery
export RECONX_DNSX_THREADS=100            # DNS resolution threads
export RECONX_HTTPX_THREADS=100           # HTTP probe threads
export RECONX_SUBFINDER_THREADS=50        # Subfinder threads

# Phase 2: Intelligence
export RECONX_RUSTSCAN_BATCH=1000         # RustScan batch size
export RECONX_NMAP_MAX_HOSTS=50           # Max hosts to deep scan
export RECONX_MIN_DISK_MB=1024            # Min disk space check

# Phase 3: Content Discovery
export RECONX_FFUF_THREADS=80             # FFUF threads
export RECONX_FEROX_THREADS=80            # Feroxbuster threads

# Phase 4: Vulnerability Scanning
export RECONX_NUCLEI_RATE_HIGH=100        # Nuclei rate limit
export RECONX_SQLMAP_THREADS=5            # SQLMap threads

# API Keys
export SHODAN_API_KEY="sk_abc123..."
export CENSYS_API_ID="..."
export SECURITYTRAILS_API_KEY="..."
```

Method 2: Config File (Future Phase A)

Currently, `config.yaml` exists but is not loaded. Future implementation will:

```
# config.yaml (not currently used, but shows intended structure)
general:
  output_dir: "output"
  database: "reconx.db"
  threads: 5
```

```yaml
    timeout: 3600

  phase1_discovery:
    enabled: true
    subdomain_tools: [sublist3r, amass, assetfinder, subfinder]
    dns_resolution:
      enabled: true
      tool: dnsx
      threads: 100
    http_validation:
      enabled: true
      tool: httpx
      threads: 50

  phase2_intel:
    port_scanning:
      enabled: true
      fast_scan: true    # RustScan
      deep_scan: true    # Nmap
    osint:
      shodan: true
      censys: true

  phase3_content:
    archive_crawling:
      enabled: true
      tools: [gau, waybackurls, hakrawler, katana]
    directory_bruteforce:
      enabled: true
      threads: 80

  phase4_vulnscan:
    nuclei:
      enabled: true
      severity: [critical, high, medium]
    xss:
      enabled: true
    sqli:
      enabled: true

  api_keys:
    shodan: "${SHODAN_API_KEY}"
    censys_id: "${CENSYS_API_ID}"
```

Scan Profiles (Future Phase B)

```
# Passive only (no risk of detection)
python3 reconx.py -t example.com --profile passive

# Standard (balanced speed/sensitivity)
```

```
python3 reconx.py -t example.com --profile standard

# Aggressive (maximum coverage, risk of triggering WAF/IDS)
python3 reconx.py -t example.com --profile aggressive

# Stealth (slow, minimal requests)
python3 reconx.py -t example.com --profile stealth

# Custom (use config.yaml settings)
python3 reconx.py -t example.com --profile custom
```

Usage Guide

Basic Scanning

```
# Scan single target
python3 reconx.py -t example.com

# Scan multiple targets
python3 reconx.py -t example.com,example.org,example.net

# Scan from file
python3 reconx.py -f targets.txt

# Custom output directory
python3 reconx.py -t example.com -o /path/to/output

# Custom database
python3 reconx.py -t example.com -d /path/to/custom.db
```

Phase-Based Scanning

```
# Run all phases (default)
python3 reconx.py -t example.com -p 1,2,3,4

# Only discovery and reconnaissance
python3 reconx.py -t example.com -p 1,2

# Only vulnerability scanning
python3 reconx.py -t example.com -p 4

# Skip phase 3 (content discovery is slow)
python3 reconx.py -t example.com -p 1,2,4
```

Multi-Threading

```
# Single target (sequential)
python3 reconx.py -t example.com -T 1

# 5 concurrent targets (default)
python3 reconx.py -t example.com,example.org,example.net -T 5

# Maximum parallelism (for large scanning)
python3 reconx.py -f 100_targets.txt -T 20
```

Querying Results

```
# List all scanned targets
python3 query.py --list

# Target summary
python3 query.py -t example.com --summary

# Show all subdomains
python3 query.py -t example.com --subdomains

# Show alive subdomains only
python3 query.py -t example.com --subdomains --alive-only

# Show vulnerabilities by severity
python3 query.py -t example.com --vulns --severity critical

# Show open ports
python3 query.py -t example.com --ports

# Show discovered URLs
python3 query.py -t example.com --urls --limit 100

# Show secrets/leaks found
python3 query.py -t example.com --leaks
```

Exporting Data

```
# Export subdomains to CSV
python3 query.py -t example.com --export subdomains -o subdomains.csv

# Export vulnerabilities to CSV
python3 query.py -t example.com --export vulnerabilities -o vulns.csv

# Export all data
python3 query.py -t example.com --export all -o report.csv
```

Resume & Retry

```
# Resume incomplete scan (currently uses phase flags)
python3 reconx.py -t example.com -p 3,4

# Wait for previous scan to complete, check status
sqlite3 reconx.db "SELECT * FROM scan_progress WHERE
target='example.com'"

# Force re-run specific phase
# (Delete completion marker from database)
sqlite3 reconx.db "UPDATE scan_progress SET phase1_done=0 WHERE
target='example.com'"
python3 reconx.py -t example.com -p 1
```

Database Schema

Complete SQLite Schema

```
-- Scan Progress Tracking
CREATE TABLE scan_progress (
    target TEXT PRIMARY KEY,
    phase1_done BOOLEAN DEFAULT 0,
    phase2_done BOOLEAN DEFAULT 0,
    phase3_done BOOLEAN DEFAULT 0,
    phase4_done BOOLEAN DEFAULT 0,
    phase1_partial BOOLEAN DEFAULT 0,
    phase2_partial BOOLEAN DEFAULT 0,
    phase3_partial BOOLEAN DEFAULT 0,
    phase4_partial BOOLEAN DEFAULT 0,
    started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tool Execution Tracking (created but not actively used)
CREATE TABLE tool_runs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    target TEXT,
    phase INTEGER,
    tool_name TEXT,
    command TEXT,
    started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    completed_at TIMESTAMP,
    exit_code INTEGER,
    status TEXT,
    output_file TEXT,
    error_file TEXT,
    duration_seconds REAL,
```

```sql
    records_found INTEGER DEFAULT 0
);

-- Subdomain Inventory
CREATE TABLE subdomains (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    target TEXT,
    host TEXT,
    ip TEXT,
    is_alive BOOLEAN DEFAULT 0,
    status_code INTEGER,
    source_tools TEXT,
    discovered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(target, host)
);
CREATE INDEX idx_subdomains_target ON subdomains(target);
Create INDEX idx_subdomains_alive ON subdomains(is_alive);

-- Open Ports
CREATE TABLE ports (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    target TEXT,
    host TEXT,
    port INTEGER,
    protocol TEXT,
    service TEXT,
    version TEXT,
    discovered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(target, host, port)
);
CREATE INDEX idx_ports_target ON ports(target);

-- Discovered URLs & Parameters
CREATE TABLE urls (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    target TEXT,
    url TEXT,
    source_tool TEXT,
    discovered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(target, url, source_tool)
);
CREATE INDEX idx_urls_target ON urls(target);

-- Secrets & Credential Leaks
CREATE TABLE leaks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    target TEXT,
    leak_type TEXT,
    url TEXT,
    info TEXT,
    severity TEXT DEFAULT 'info',
    discovered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
CREATE INDEX idx_leaks_target ON leaks(target);

-- Discovered Vulnerabilities
CREATE TABLE vulnerabilities (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    target TEXT,
    host TEXT,
    tool TEXT,
    severity TEXT,
    name TEXT,
    info TEXT,
    cve TEXT,
    discovered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_vulns_target ON vulnerabilities(target);
CREATE INDEX idx_vulns_severity ON vulnerabilities(severity);

-- Infrastructure & OSINT Data
CREATE TABLE infrastructure (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    target TEXT,
    host TEXT,
    ip TEXT,
    asn TEXT,
    org TEXT,
    cloud_provider TEXT,
    cdn TEXT,
    technologies TEXT,
    discovered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(target, host)
);

-- Acquisition/Company Relationships (for future use)
CREATE TABLE acquisitions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    target TEXT,
    company TEXT,
    domain TEXT,
    discovered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(target, company, domain)
);
```

Query Examples

```sql
-- Count subdomains per target
SELECT target, COUNT(*) as count FROM subdomains GROUP BY target;

-- Count alive hosts
SELECT target, COUNT(*) as alive_count
FROM subdomains
```

```sql
WHERE is_alive = 1
GROUP BY target;

-- Top severity vulnerabilities
SELECT target, severity, COUNT(*) as count
FROM vulnerabilities
GROUP BY target, severity
ORDER BY target, severity DESC;

-- Services detected
SELECT DISTINCT service, COUNT(*) as count
FROM ports
WHERE service IS NOT NULL
GROUP BY service;

-- URLs by tool source
SELECT source_tool, COUNT(*) as count
FROM urls
GROUP BY source_tool;

-- Critical findings
SELECT target, name, CVE, COUNT(*) as instances
FROM vulnerabilities
WHERE severity = 'critical'
GROUP BY target, name
ORDER BY instances DESC;
```

Four-Phase Reconnaissance Engine

Phase 1: Discovery & Enumeration

Objective: Identify all assets (subdomains, IP ranges, acquisitions)

Tools Executed:

Category	Tools
Horizontal	Whois, Amass Intel, getSubsidiaries
Subdomain Enum	Sublist3r, Amass Enum, Assetfinder, Subfinder, SubDominator, crt.sh, CertSpotter, ct-monitor, SecurityTrails API, Dnsbruter, DnsProber
Cloud/ASN	ASNMap, MapCIDR, cloud_enum, s3scanner, GobBlob, GCPBucketBrute
Validation	DNSx (resolution), HTTPx (probing)

Example Output:

```
output/example_com/phase1_discovery/
├── whois.txt                          (domain registration info)
```

```
├── all_subdomains.txt                  (5000+ subdomains)
├── passive_subdomains.txt              (from CT sources)
├── active_subdomains.txt               (from bruteforce)
├── resolved_subdomains.txt             (with IPs)
├── alive_hosts.txt                     (HTTP responds)
├── httpx_alive.json                    (status codes, headers)
├── dnsx_resolved.json                  (DNS records)
├── temp_subdomains/                    (per-tool outputs)
│   ├── amass.txt
│   ├── assetfinder.txt
│   ├── subfinder.txt
│   ├── sublist3r.txt
│   └── ...
└── cloud/                              (cloud exposure check)
    ├── s3scanner_results.txt
    ├── cloud_enum_results.txt
    └── ...
```

Data Inserted to DB:

- ✅ Subdomains (with source tools)
- ✅ Resolved IPs
- ✅ Alive/Dead status
- ❌ Whois information
- ❌ Cloud exposure data

Key Statistics:

- Typical result: 1,000-10,000 subdomains per domain
- Discovery time: 15-45 minutes (depending on domain size)

Phase 2: Intelligence & Infrastructure

Objective: Map network topology, detect services, find sensitive data

Tools Executed:

Category	Tools
Port Scanning	RustScan (fast), Nmap (detailed service detection)
Validation	SubProber
OSINT	Shodan, ShodanX, Censys, Google Dorking, whois ASN lookup
Takeover	Subjack, SubOver
Leaks	Gitleaks, GitHunt, TruffleHog, git-secrets

Example Output:

```
output/example_com/phase2_intel/
├── ports/
│   ├── nmap_all.xml                    (service versions, OS)
│   ├── rustscan_ports.txt              (quick port discovery)
│   └── scan_hosts.txt                  (validated targets)
├── osint/
│   ├── shodan_results.json             (service info from Shodan)
│   ├── censys_results.json             (certificate intelligence)
│   ├── asn_info.txt                    (ASN lookups)
│   └── google_dorking.txt              (sensitive endpoints)
├── takeover/
│   └── subjack_results.txt             (vulnerable to takeover)
└── leaks/
    ├── gitleaks_*.json                 (secrets found in git)
    ├── trufflehog_*.json               (credential patterns)
    └── github_results.txt              (public GitHub leaks)
```

Data Inserted to DB:

- ✅ Open ports (with service versions)
- ✅ Takeover vulnerabilities
- ✅ Git leaks
- ❌ Shodan/Censys data
- ❌ ASN information

Key Statistics:

- Typical result: 20-100 open ports per asset
- Common services: 80 (HTTP), 443 (HTTPS), 22 (SSH), 3389 (RDP)
- Discovery time: 30-60 minutes

Phase 3: Deep Web & Content Discovery

Objective: Map web application surface, find hidden endpoints and parameters

Tools Executed:

Category	Tools
Archive Crawling	GAU, Wayback Machine API, SpideyX, Gospider, Hakrawler, Katana
Directory Brute	FFUF, Feroxbuster, Dirsearch
JavaScript Analysis	LinkFinder, SecretFinder, JSScanner, SpideyX JS Scraper
Pastebin Monitoring	PasteHunter, Pastebin API, pbin
API Discovery	Newman, KiteRunner, Arjun

Example Output:

```
output/example_com/phase3_content/
├── urls/
│   ├── all_urls.txt                    (100k+ discovered URLs)
│   ├── javascript_files.txt            (JS files)
│   ├── gau.txt                         (wayback + archives)
│   ├── waybackurls.txt                 (archive.org results)
│   ├── katana.txt                      (live crawling)
│   ├── gospider.txt                    (spider results)
│   └── ...
├── bruteforce/
│   ├── ffuf_all.json                   (directory scan results)
│   ├── feroxbuster_all.txt             (recursive bruteforce)
│   ├── brute_targets.txt               (hosts tested)
│   └── wordlist.txt                    (paths tested)
├── javascript/
│   ├── all_javascript.txt              (JS file URLs)
│   ├── linkfinder_endpoints.txt        (API endpoints)
│   ├── secretfinder_secrets.txt        (API keys, tokens)
│   └── ...
└── pastebin/
    ├── pastes.txt                      (related pastes)
    └── extracted_secrets.txt           (found credentials)
```

Data Inserted to DB:

- ✅ URLs (from archive tools)
- ✅ JavaScript secrets/endpoints
- ❌ Feroxbuster results
- ❌ Dirsearch results
- ❌ Pastebin entries

Key Statistics:

- Typical result: 10,000-100,000+ URLs per domain
- JS files: 100-1000+
- API endpoints found: 50-500+
- Discovery time: 45-120 minutes

Phase 4: Vulnerability Scanning

Objective: Identify security weaknesses in discovered assets

Tools Executed:

Category	Tools
Template-Based	Nuclei (all templates, severity-grouped)
Web Vulns	Dalfox (XSS), SQLMap (SQLi), Corsy (CORS), XSStrike

Category	Tools
CMS Scanning	WPScan, CMSmap, Wapiti
SSL/TLS	testssl.sh, SSLyze
Comprehensive	Nikto, Skipfish, Retire.js

Example Output:

```
output/example_com/phase4_vulnscan/
├── nuclei/
│   ├── nuclei_critical_high.json      (most severe)
│   ├── nuclei_medium.json             (medium findings)
│   ├── nuclei_low_info.json           (low severity)
│   ├── nuclei_cve.json                (CVE-specific)
│   ├── nuclei_misconfig.json          (config issues)
│   └── nuclei_all.json                (combined)
├── xss/
│   ├── dalfox_results.txt             (XSS findings)
│   └── param_urls.txt                 (tested parameters)
├── sqli/
│   ├── sqlmap_results.txt             (SQL injection)
│   └── tested_urls.txt
├── cors/
│   └── corsy_results.txt              (CORS misconfig)
├── ssl/
│   └── testssl_results.json           (SSL/TLS issues)
└── scan_urls.txt                      (all tested URLs)
```

Data Inserted to DB:

- ✅ Nuclei findings (by severity)
- ✅ Dalfox (XSS) findings
- ✅ SQLMap (SQLi) findings
- ✅ Corsy (CORS) findings
- ❌ Nikto results
- ❌ WPScan results
- ❌ SSL/TLS details

Key Statistics:

- Typical result: 10-1000+ vulnerabilities per domain (depends on size)
- Critical findings: 0-10 (if present, urgent action)
- High findings: 5-50 (require remediation)
- Discovery time: 60-180 minutes

API Documentation (Planned)

Phase A: RESTful API (Planned Q2 2026)

Complete REST API for programmatic access to all ReconX functions.

Base URL

```
http://localhost:8000/api/v1
```

Authentication

```
Header: X-API-Key: sk_prod_abc123xyz789
```

Scan Management Endpoints

Create Scan

```
POST /scans
Content-Type: application/json

{
  "target": "example.com",
  "phases": [1,2,3,4],
  "threads": 5
}

Response:
{
  "scan_id": "uuid-123",
  "target": "example.com",
  "status": "pending",
  "created_at": "2026-02-10T10:00:00Z"
}
```

List Scans

```
GET /scans?status=running&limit=50

Response:
{
  "total": 234,
  "scans": [
    {
      "scan_id": "uuid-1",
```

```
      "target": "target1.com",
      "status": "running",
      "progress": 45,
      "phases": {
        "1": true,
        "2": false,
        "3": false,
        "4": false
      }
    }
  ]
}
```

Get Scan Details

```
GET /scans/{scan_id}

Response:
{
  "scan_id": "uuid-123",
  "target": "example.com",
  "status": "completed",
  "phases": {
    "1": {"done": true, "duration": 1800},
    "2": {"done": true, "duration": 2400},
    "3": {"done": true, "duration": 3600},
    "4": {"done": true, "duration": 2700}
  },
  "statistics": {
    "subdomains": 5234,
    "alive_hosts": 892,
    "urls": 45230,
    "ports": 234,
    "vulnerabilities": 156,
    "critical_vulns": 3,
    "high_vulns": 12
  }
}
```

Data Retrieval Endpoints

Subdomains

```
GET /targets/{target}/subdomains?alive=true&limit=100&offset=0

Response:
{
```

```
  "total": 5234,
  "data": [
    {
      "subdomain": "api.example.com",
      "ip": "203.0.113.45",
      "alive": true,
      "status_code": 200,
      "source_tools": ["subfinder", "amass"],
      "discovered_at": "2026-02-10T09:30:00Z"
    }
  ]
}
```

Vulnerabilities

```
GET /targets/{target}/vulnerabilities?severity=critical,high&limit=100

Response:
{
  "total": 156,
  "critical": 3,
  "high": 12,
  "medium": 45,
  "low": 96,
  "data": [
    {
      "vuln_id": "vuln-123",
      "host": "admin.example.com",
      "name": "SQL Injection in /login",
      "severity": "critical",
      "tool": "sqlmap",
      "cve": "CVE-2024-12345",
      "info": "Vulnerable parameter: username",
      "discovered_at": "2026-02-10T14:00:00Z"
    }
  ]
}
```

Open Ports

```
GET /targets/{target}/ports?filter_port=22,80,443

Response:
{
  "total": 234,
  "data": [
    {
```

```
      "host": "web.example.com",
      "port": 443,
      "protocol": "tcp",
      "service": "https",
      "version": "nginx/1.25.1"
    }
  ]
}
```

URLs

```
GET /targets/{target}/urls?keyword=admin&limit=100

Response:
{
  "total": 1234,
  "data": [
    {
      "url": "https://example.com/admin/login",
      "source_tool": "waybackurls",
      "status_code": 200,
      "discovered_at": "2026-02-10T13:00:00Z"
    }
  ]
}
```

Report Endpoints

Generate Report

```
POST /targets/{target}/reports

{
  "format": "pdf",  # pdf, html, json, csv
  "include_sections": ["executive_summary", "detailed_findings",
"remediation"],
  "filter_severity": "high+"
}

Response:
{
  "report_id": "report-456",
  "format": "pdf",
  "status": "generating",
  "download_url": "/reports/report-456/download"
}
```

Export Data

```
GET /targets/{target}/export?format=csv&data_type=vulnerabilities

Response: CSV file download
```

Analytics Endpoints

Dashboard Stats

```
GET /dashboard/stats

Response:
{
  "total_targets": 234,
  "active_scans": 5,
  "completed_scans": 3421,
  "total_vulnerabilities": 45678,
  "critical": 123,
  "high": 567,
  "new_assets_last_week": 892,
  "new_vulns_last_week": 234
}
```

Web Dashboard (Planned)

Phase A: Interactive Web UI (Planned Q2 2026)

Modern, responsive web interface for ReconX.

Dashboard Home

- Real-time scan progress
- KPI cards: Total domains, active scans, critical vulns, new assets this week
- Chart: Vulnerabilities trend (last 30 days)
- Quick actions: Start scan, view reports, export data

Scan Management Page

- Table of all scans with filters
- Columns: Target, Status, Phase, Progress, Started, Completed, Actions
- Bulk operations: Start scan, pause, resume, delete
- Search and filter by target, status, date range

Target Details Page

- Subdomains list (searchable, filterable by alive/IP)
- Ports table (sort by port, service, version)
- URLs list (paginated, filter by source)
- Vulnerabilities (severity-based colors, drill-down)
- Infrastructure metadata (ASN, CDN, tech stack)

Vulnerability Explorer

- Interactive table with all findings
- Severity-based color coding
- Filter/search by: name, CVE, host, tool, severity
- Drill-down: Click vulnerability to see details + remediation
- Status tracking: Mark as open/in-progress/resolved/false-positive

Attack Surface Map

- Graph visualization: Domain → Subdomains → Ports → Services → Vulns
- Interactive nodes: Click to expand, zoom, pan
- Color coding: Green (safe), Yellow (warnings), Red (critical)
- Topology view: Show relationships between assets

Reporting

- Pre-built templates: Executive Summary, Detailed Report, Compliance Report
- Custom report builder: Select sections, filters, style
- PDF/HTML export
- Email delivery scheduled reports

Settings

- API Key management
- Notification channels (Slack, Discord, Email)
- Scan profiles and policies
- User management (Phase C)
- Integrations (Jira, ServiceNow) (Phase C)

Roadmap: 4-Phase Development Plan

Phase A: MVP Market Release (2-3 weeks)

Goal: Production-ready SaaS launch

Deliverables:

- ✅ REST API (FastAPI) - All 15+ endpoints
- ✅ Web Dashboard (React/Next.js) - All 6 pages
- ✅ Report Generation - PDF, HTML, CSV

- ✅ Docker Support - docker-compose.yml
- ✅ Config Loading - YAML support fully functional
- ✅ Fix bugs - FFUF parsing, HTTPx updates, config.yaml loading

Technical:

- API rate limiting
- Basic authentication (static API keys)
- Swagger/OpenAPI documentation
- Docker health checks
- Comprehensive logging

Testing:

- API endpoint tests (FastAPI TestClient)
- Dashboard E2E tests (Cypress)
- Load test: 100 concurrent requests

Estimated Effort: 160-200 hours

Success Metrics:

- API responds in <1s
- Dashboard loads in <3s
- Docker deploys cleanly

Phase B: Competitive Features (3-4 weeks)

Goal: Match commercial ASM platform capabilities

Deliverables:

- ✅ Scheduled Scans (APScheduler) - Cron-based recurring scans
- ✅ Continuous Monitoring - New asset alerts, DNS change alerts
- ✅ Notifications - Slack, Discord, Email, Teams, Custom Webhooks
- ✅ Alert Rules Engine - Custom triggers and actions
- ✅ Scope Management - In/out of scope definitions
- ✅ Rate Limiting - Per-target throttling
- ✅ Auto-tagging - Cloud provider, CDN, WAF detection
- ✅ Vulnerability Deduplication - De-duplicate across tools
- ✅ Vulnerability Tracking - Status, assignment, due dates
- ✅ CVSS Enrichment - Fetch official scores

Database Updates:

- New tables: schedules, alert_rules, alert_history, scope_definitions
- Enhanced vulnerabilities table: status, assigned_to, due_date, fingerprint

Testing:

- Scheduler unit tests
- Notifier integration tests (mock webhooks)
- Scope matcher tests
- Deduplication logic tests

Estimated Effort: 200-240 hours

Success Metrics:

- Scheduled scans run on time
- Alerts deliver < 30 seconds
- Deduplication reduces vuln count by 40-60%

Phase C: Enterprise Edition (4-5 weeks)

Goal: Multi-team, secure collaboration

Deliverables:

- ✅ Multi-Tenancy - Complete org/user/workspace isolation
- ✅ Authentication - JWT + API Keys + Refresh tokens
- ✅ RBAC - Admin, Analyst, Viewer roles + custom roles
- ✅ Audit Logging - All user actions logged
- ✅ Jira Integration - Auto-create/update issues, bi-directional sync
- ✅ ServiceNow Integration - Incident management
- ✅ Compliance Frameworks - OWASP Top 10, NIST, PCI-DSS, CIS
- ✅ Policy Engine - Auto-policy evaluation, violation tracking
- ✅ SSO/LDAP - OAuth2 (Google, GitHub), SAML, LDAP

Database Updates:

- New tables: organizations, users, api_keys, audit_logs, compliance_policies, compliance_results, integrations

Testing:

- RBAC permission matrix tests
- Multi-org isolation tests (org A can't see org B data)
- Jira sync tests
- SAML SSO tests

Estimated Effort: 240-300 hours

Success Metrics:

- Multi-tenant database < 10ms query time
- 99.9% uptime for API
- Jira issues create/update in < 5 seconds

Phase D: Scale & Advanced (4-6 weeks)

Goal: Enterprise-scale infrastructure

Deliverables:

- ✅ PostgreSQL Migration - Full support, with data migration script
- ✅ Distributed Workers - Celery + Redis task queue
- ✅ Plugin System - Custom tools, parsers, enrichers, notifiers
- ✅ Wappalyzer Integration - Tech stack fingerprinting
- ✅ Prometheus Metrics - Full observability
- ✅ Grafana Dashboards - Performance monitoring
- ✅ ELK Stack Integration - Log aggregation
- ✅ Cost Tracking - Usage monitoring and billing
- ✅ Kubernetes Support - Helm charts for K8s deployment
- ✅ Load Balancing - Horizontal scaling of workers

Architecture Changes:

- Controller-Worker model
- Task queue for distributed execution
- Caching layer (Redis)
- CDN for static assets
- Database connection pooling

Testing:

- Load test: 1000 concurrent scans
- Chaos engineering: Test worker failure scenarios
- Plugin system: Test custom plugin execution

Estimated Effort: 280-360 hours

Success Metrics:

- Handle 10,000+ targets in production
- 99.99% uptime
- Scan <100 hosts in <10 minutes

Use Cases

1. Startup Security Program

Scenario: Early-stage SaaS startup establishing first security program

Flow:

1. Run ReconX on startup's domain
2. Get baseline of discovered assets
3. Find exposed databases, secrets in git, weak SSL

4. Create Jira issues for each finding
5. Weekly scheduled scans to track progress
6. Export monthly report for compliance

Value: Discover 80% of real attack surface in <2 hours

---

2. M&A Due Diligence

Scenario: Acquiring company performing technical due diligence on target

Flow:

1. Scan target company's entire digital footprint in parallel
2. Export detailed report with vulnerability breakdown
3. Benchmark against industry standards
4. Identify shadow IT, hidden infrastructure
5. Build risk assessment

Value: 3-day assessment instead of 3-week manual review

---

3. Continuous Security Monitoring

Scenario: Security team monitoring 500+ domains for continuous exposure

Flow:

1. Configure ReconX with all corporate domains
2. Schedule daily/weekly scans
3. Alerts when new subdomains discovered
4. Alerts when new critical vulns found
5. Monthly compliance report to board
6. Metrics: New assets, resolved vulns, critical findings

Value: 24/7 attack surface visibility, catch threats faster than attackers

---

4. Compliance & Audit Readiness

Scenario: Health care org preparing for HIPAA audit

Flow:

1. Run ReconX against all patient-facing systems
2. Apply HIPAA policy checks
3. Generate compliance report showing:
    - All assets inventoried
    - No SSL/TLS issues (encryption in transit)
    - No open databases
    - No default credentials

4. Export evidence for auditors

Value: Demonstrate comprehensive inventory + security posture

---

5. Red Team Assessment

Scenario: Penetration tester gathering intel for red team exercise

Flow:

1. Run all 4 phases on target
2. Export comprehensive asset inventory
3. Identify weak points: unpatched services, weak SSL, known CVEs
4. Plan red team approach based on data
5. Use ReconX findings as "agreed-upon scope"

Value: Faster, data-driven red team planning

---

6. Incident Response

Scenario: Security team responding to breach notification

Flow:

1. Immediately scan breached domain with ReconX
2. Find compromised servers, backdoors
3. Identify blast radius: what other assets affected
4. Export for forensics team
5. Track remediation progress with update scans

Value: Rapid asset inventory and threat scope determination

Best Practices

Scanning Strategy

Passive First

```
# Low risk, stealthy scanning
python3 reconx.py -t target.com --profile passive
# Only phases 1-2 (discovery, port scan), no active probing
```

Graduated Approach

```
# Day 1: Passive discovery (1-2 hours)
python3 reconx.py -t target.com -p 1,2
```

```
# Day 2: Content discovery (2-4 hours, can trigger WAF)
python3 reconx.py -t target.com -p 3

# Day 3: Vulnerability scanning (2-4 hours, active testing)
python3 reconx.py -t target.com -p 4
```

Scope Management

```
# List out-of-scope assets FIRST
# Only scan authorized systems
# Use scope exclusions in config (Phase B):

export RECONX_EXCLUDE_DOMAINS="internal.example.com,dev.example.com"
export RECONX_EXCLUDE_IPS="10.0.0.0/8"
```

Performance Optimization

Single Large Domain

```
# Use fewer workers to manage tool noise
python3 reconx.py -t fortessecorp.com -T 1 -p 1,2,3,4
```

Multiple Smaller Domains

```
# Parallelize across disjoint domains
python3 reconx.py -f targets.txt -T 10 -p 1,2,3,4
```

Memory Constraints

```
# Reduce thread counts
export RECONX_DNSX_THREADS=50     # Default 100
export RECONX_HTTPX_THREADS=20    # Default 100
export RECONX_FFUF_THREADS=20     # Default 80
```

Result Quality

Deduplication

```
# Multiple tools find same vuln
# Phase B API will show deduplicated results
# Check "duplicates_count" field to understand redundancy
```

False Positives

```
# Review high-confidence tool outputs first:
# 1. Nuclei (high accuracy)
# 2. Nmap (port configuration fact)
# 3. Dalfox (XSS confirmation rate ~70%)
# 4. Review less accurate tools manually

python3 query.py -t target.com --export nuclei_only -o nuclei.csv
```

Data Freshness

```
# Older results stale after:
# - Subdomains: Weekly (new assets emerge)
# - Ports: Monthly (service changes less frequent)
# - Vulns: Weekly (patches, configs change)
# - URLs: Monthly (content stable unless dynamic site)

# Schedule re-scans:
# Weekly: Phases 1-2 (discovery + ports)
# Monthly: Phases 3-4 (content + vulns)
```

Cost Optimization

API Key Efficiency

```
# Shodan/Censys/SecurityTrails queries are metered
# Batch requests when possible

# Phase B will include quota tracking:
python3 query.py --api-usage
# Shows: "Shodan: 45/100 queries used this month"
```

Tool Elimination

```
# If scanning thousands of targets, consider disabling:
# - Slow tools (CertSpotter API - rate limited)
# - Redundant tools (multiple subdomain tools find same results)

# Phase B config profiles:
python3 reconx.py -t target.com --profile fast
# Disables: slow certificate tools, redundant subdom tools
# Keeps: core discovery + critical vulnerability scanners
```

Troubleshooting

Common Issues

Problem	Solution
Tools not found	`sudo bash install.sh && source ~/.bashrc`
SQLite "locked" error	`sqlite3 reconx.db "PRAGMA wal_checkpoint(TRUNCATE);"`
No results for Phase X	Check `output/target/phaseX/` for raw tool outputs
API timeout	Increase `RECONX_PHASE_TIMEOUT` env var
Missing subdomains	Enable multiple tools, check DNS working
No ports found	Verify target returns HTTP responses
Missing vulnerabilities	Confirm tools installed, review tool outputs

Debug Mode

```
# Enable verbose output
export RECONX_DEBUG=1
python3 reconx.py -t target.com

# Check tool availability
bash modules/01_discovery.sh target.com output/test 2>&1 | head -50

# Review database directly
sqlite3 reconx.db
sqlite> SELECT * FROM vulnerabilities WHERE target='target.com' LIMIT
10;

# Check raw tool outputs
ls -lR output/target_com/
cat output/target_com/phase1_discovery/all_subdomains.txt | wc -l
```

Performance Issues

```
# Slow DNS resolution
export RECONX_DNSX_TIMEOUT=30
export RECONX_DNSX_THREADS=200

# Slow HTTP probing
export RECONX_HTTPX_TIMEOUT=20
export RECONX_HTTPX_THREADS=200
```

```
# Reduced memory usage (slower)
export RECONX_DNSX_THREADS=20
export RECONX_HTTPX_THREADS=20
```

Contributing

How to Contribute

1. **Report Issues**
   - Describe the problem
   - Include target domain (sanitized)
   - Provide ReconX version and output
2. **Suggest Features**
   - Review roadmap above
   - Suggest within Phase A-D framework
   - Provide use case
3. **Submit Code**
   - Fork repository
   - Create feature branch
   - Submit pull request with:
     - Description
     - Test coverage
     - Documentation

Development Environment

```
# Set up dev environment
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install pytest pytest-cov black flake8 mypy

# Run tests (Phase A will have full test suite)
pytest tests/ -v --cov

# Code quality
black . && flake8 . && mypy
```

License

ReconX is open-source under the MIT License.

Legal: Use only on systems you own or have explicit written authorization to test. Unauthorized scanning may violate laws in your jurisdiction. The authors assume no liability.

Support & Community

- **Issues:** GitHub Issues tracker
- **Discussions:** GitHub Discussions
- **Email:** support@reconx.dev (future)
- **Documentation:** This file + wiki (future)

Changelog

Version 1.0 (Current - February 2026)

- ✅ Core orchestration engine completed
- ✅ 4-phase modular execution
- ✅ 50+ tool orchestration
- ✅ SQLite storage with WAL mode
- ✅ CLI querying and export
- 🔄 API and Dashboard in development (Phase A)

Last Updated: February 10, 2026
Next Update: After Phase A completion (estimated April 2026)